INTEGRATED SMART SYSTEM CAPSTONE

Curiosity Kickoff

- What does “integrated” mean?
- What makes a system “smart”?
- Can one system do many tasks?

Concept Connection

An Integrated Smart System Capstone is a comprehensive project that combines multiple sensors, logic circuits, and control systems into one unified solution. It demonstrates how different technologies work together to create an intelligent system capable of sensing, processing, and responding automatically for real-world applications.

1.Watt SMD LED

I’m flat and powerful. Just one of me can light up your whole table like a torch.I don’t like wasting energy, so I shine super bright without getting too hot,You’ve already met me — I live in your mobile flashlight!

2.Buzzer

A sound-making device that works when electricity makes a thin metal plate vibrate rapidly inside. The vibration creates buzzing or beeping sounds. Some buzzers can even change frequency to play simple tunes or melodies.

3.LDR Sensor :

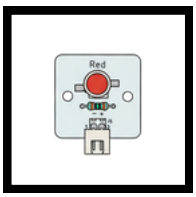
A light sensor whose resistance changes depending on light intensity. Lower resistance in bright light, higher in darkness.
Pinout: 4 Pins - VCC (+), Ground (-), Digital Pin (D0) & Analog Pin (A0)

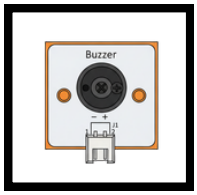
4**.IR (Infrared) Sensor :**

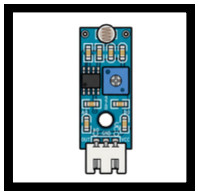
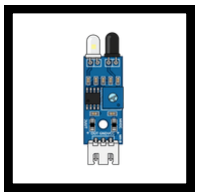
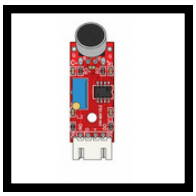
Detects infrared light, which is invisible to human eyes,Often used for obstacle detection or remote control.
Pinout: 3 pins - VCC: Power, GND: Ground & OUT: Digital signal

5.Sound Sensor

Has a small microphone inside that converts sound waves (claps, voices, knocks) into electrical signals. The circuit then compares the loudness to a set level and sends a digital or analog signal to Arduino.





Beginner: Sensor Timer System

Class Explanation

A Sensor Timer System uses sensors along with timing circuits to control devices for a specific duration. When a sensor is triggered, the system activates an output like an LED or buzzer for a set time, making it useful in automation, security, and delay-based electronic applications.

Materials Required

- Arduino with attached shield
- 1 × IR sensor
- 1 × 7 Segment display
- RMC wires
- USB cable

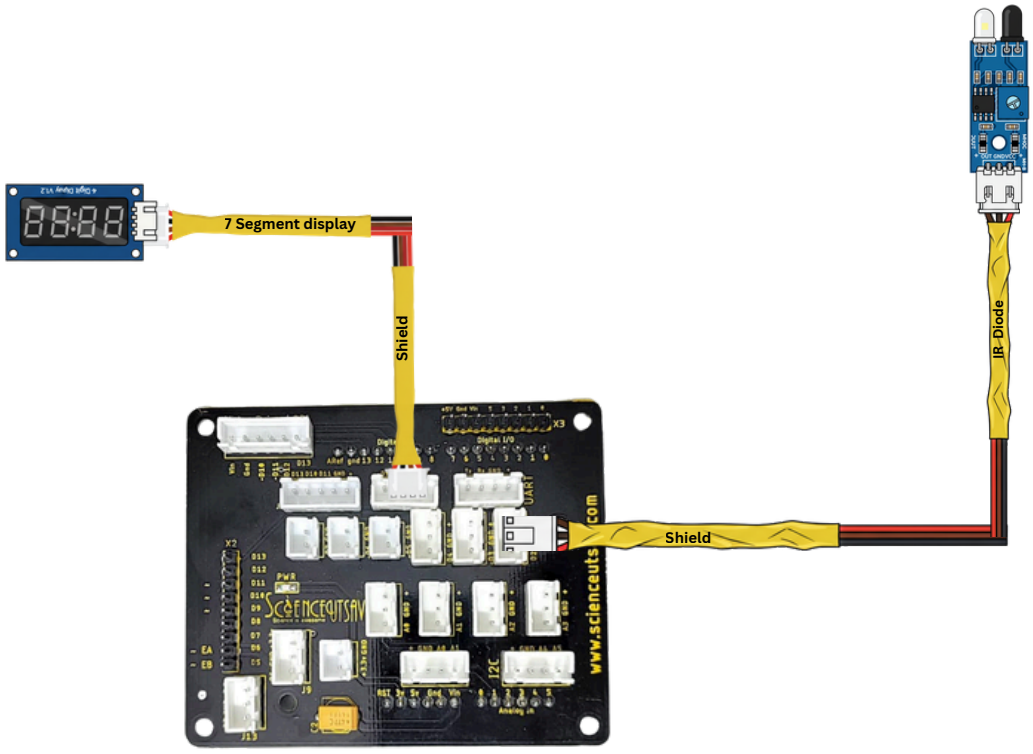
How to Connect

Use RMC wires to connect :

- 1 × IR sensor → Arduino pin D2 and GND
- 1 × 7 segment display → Arduino pin D8 D9 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <TM1637Display.h>

#define CLK 8
#define DIO 9
#define IR_PIN 2    // IR sensor on D2

TM1637Display display(CLK, DIO);

int count = 0;
int lastIRState = HIGH;

void setup() {
  pinMode(IR_PIN, INPUT);
  display.setBrightness(7);
  display.showNumberDec(0);
}

void loop() {
  int irState = digitalRead(IR_PIN);

  // Detect object (LOW = detected for most IR sensors)
  if (irState == LOW && lastIRState == HIGH) {
    count++;
    if (count == 10) count = 0;

    display.showNumberDec(count);
    delay(200); // debounce
  }

  lastIRState = irState;
}
```

Code Explanation

- In this Arduino program, an IR sensor is connected to pin 2 (IR_PIN) and a TM1637 4-digit display is connected to pins 8 (CLK) and 9 (DIO). The display is used to show a count.

- The setup() function runs once when the Arduino starts. It sets the IR sensor as INPUT, sets the display brightness using setBrightness(), and initializes the display to 0.

- The loop() function runs continuously. It reads the IR sensor value using digitalRead() and stores it in irState.

- When an object is detected (change from HIGH to LOW), the count value increases by 1. The updated count is displayed using showNumberDec(). If the count reaches 10, it resets back to 0.

- A small delay() is added to avoid multiple counts for a single detection.

So overall, the system counts objects detected by the IR sensor and displays the count on the display.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Sensor Timer System Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <TM1637Display.h>

#define CLK 8
#define DIO 9
#define IR_PIN 2     // IR sensor on D2
#define LED_PIN 5    // LED on D5

TM1637Display display(CLK, DIO);

int count = 0;
int lastIRState = HIGH;

void setup() {
  pinMode(IR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  display.setBrightness(7);
  display.showNumberDec(0);

  digitalWrite(LED_PIN, LOW);
}

void loop() {
  int irState = digitalRead(IR_PIN);

  if (irState == LOW && lastIRState == HIGH) {
    count++;
    if (count == 10) count = 0;

    display.showNumberDec(count);

    digitalWrite(LED_PIN, HIGH); // LED ON
    delay(200);                  // visible blink
    digitalWrite(LED_PIN, LOW);  // LED OFF
  }

  lastIRState = irState;
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Sensor Timer System Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Sensor Logic Challenge

Class Explanation

A Sensor Logic Challenge is a project that involves combining multiple sensors with logic circuits to solve specific tasks. It tests how inputs from different sensors are processed to control outputs like LEDs or alarms, helping develop problem-solving and circuit design skills in electronics.

Materials Required

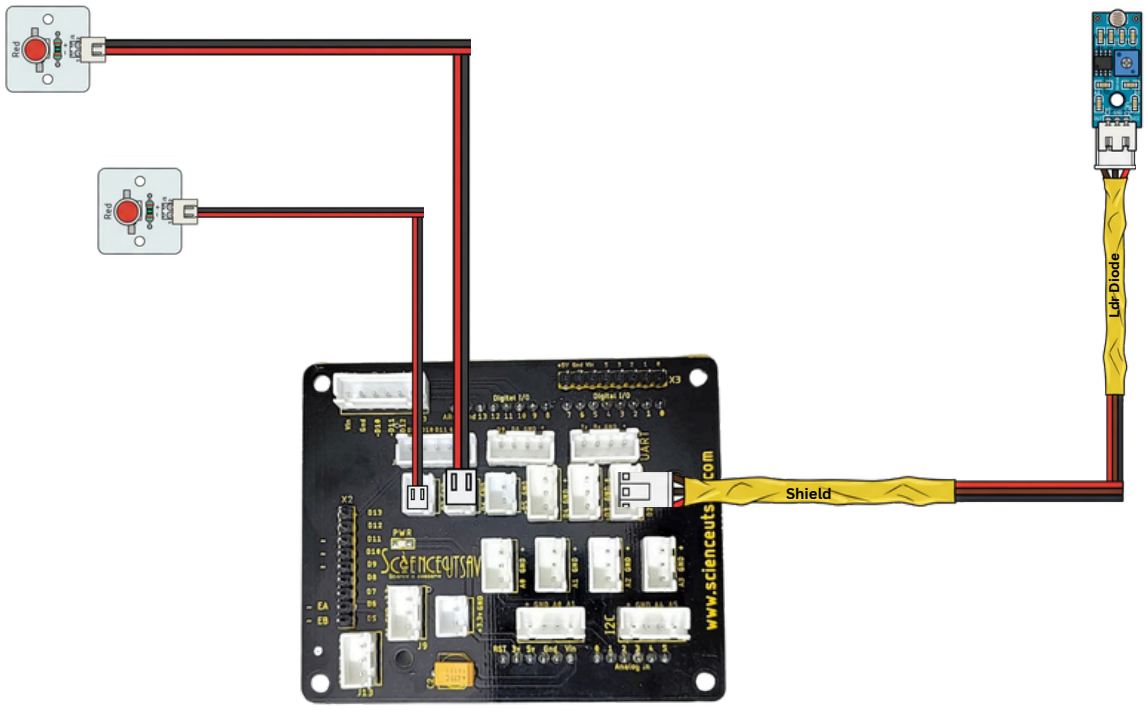
- Arduino with attached shield
- 1 × LDR sensor
- 1 × Red 1 Watt SMD LED
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × LDR sensor→ Arduino pin D2 and GND
- 1 × Red 1 Watt SMD LED → Arduino pin D6 and GND
- 1 × Red 1 Watt SMD LED → Arduino pin D7 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define LDR_PIN 2 // LDR sensor connected to D2
#define LED_PIN1 6 // LED connected to D6
#define LED_PIN2 7 // LED connected to D7 (works opposite to D6)

void setup() {
 pinMode(LDR_PIN, INPUT);
 pinMode(LED_PIN1, OUTPUT);
 pinMode(LED_PIN2, OUTPUT);
 digitalWrite(LED_PIN1, LOW);
 digitalWrite(LED_PIN2, LOW);
}

void loop() {
 int ldrstate = digitalRead(LDR_PIN);

 if (ldrstate == HIGH) {
  digitalWrite(LED_PIN1, HIGH); // Turn LED at D6 ON
  digitalWrite(LED_PIN2, LOW); // Turn LED at D7 OFF (opposite behavior)
 } else {
  digitalWrite(LED_PIN1, LOW); // Turn LED at D6 OFF
  digitalWrite(LED_PIN2, HIGH); // Turn LED at D7 ON (opposite behavior)
 }
}
```

Code Explanation

- In this Arduino program, an LDR sensor is connected to pin 2 (LDR_PIN), and two LEDs are connected to pins 6 (LED_PIN1) and 7 (LED_PIN2). The second LED works opposite to the first LED.

- The setup() function runs once when the Arduino starts. It sets the LDR as INPUT and both LEDs as OUTPUT using pinMode(). Initially, both LEDs are turned OFF.

- The loop() function runs continuously. It reads the LDR value using digitalRead() and stores it in ldr state.

- If light is detected (HIGH), the LED on pin 6 turns ON and the LED on pin 7 turns OFF. If there is no light (LOW), the LED on pin 6 turns OFF and the LED on pin 7 turns ON.

So overall, the Arduino controls two LEDs in opposite ways based on light intensity.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Sensor Logic Challenge with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define LDR_PIN 2     // LDR sensor on D2
#define BUZZ1_PIN 6   // Buzzer 1 on D6
#define BUZZ2_PIN 7   // Buzzer 2 on D7 (opposite)

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(BUZZ1_PIN, OUTPUT);
  pinMode(BUZZ2_PIN, OUTPUT);

  digitalWrite(BUZZ1_PIN, LOW);
  digitalWrite(BUZZ2_PIN, LOW);
}

void loop() {
  int ldrstate = digitalRead(LDR_PIN);

  if (ldrstate == HIGH) {
    digitalWrite(BUZZ1_PIN, HIGH); // Buzzer1 ON
    digitalWrite(BUZZ2_PIN, LOW);  // Buzzer2 OFF
  } else {
    digitalWrite(BUZZ1_PIN, LOW);  // Buzzer1 OFF
    digitalWrite(BUZZ2_PIN, HIGH); // Buzzer2 ON
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:
the basic idea of Sensor Logic Challenge with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: Integrated Smart System

Class Explanation

An Integrated Smart System using Sound Sensor combines a sound sensor with control and logic circuits to create an intelligent system. It detects sound signals and processes them to control outputs like LEDs or alarms, enabling automated responses for applications such as security, monitoring, and interactive electronics.

Materials Required

- Arduino with attached shield
- 1 × Sound sensor
- 1 × White 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

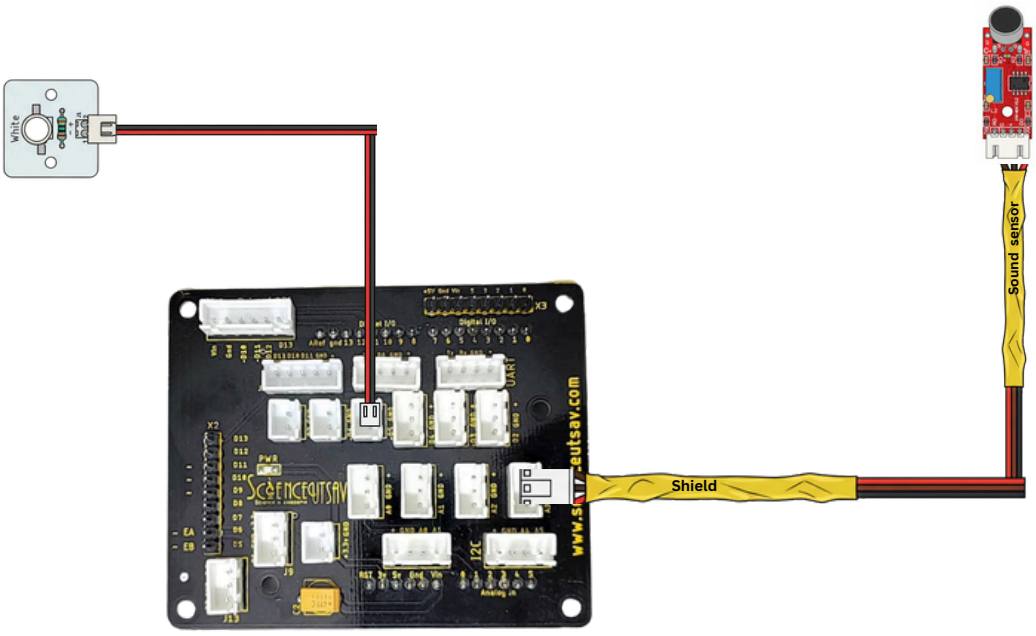
How to Connect

Use RMC wires to connect :

- 1 × Sound sensor → Arduino pin A3 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define SOUND_PIN A3 // Analog output pin from sound sensor
#define LED_PIN 5   // LED connected to D5

void setup() {
 Serial.begin(9600); // Start the Serial Monitor
 pinMode(LED_PIN, OUTPUT);
}

void loop() {
 int soundValue = analogRead(SOUND_PIN); // Read the sound sensor value (0-1023)
 Serial.print("Sound value: ");
 Serial.println(soundValue);      // Print the sound value

 int brightness = map(soundValue, 0, 1023, 0, 255); // Map to PWM range
 analogWrite(LED_PIN, brightness);         // Adjust LED brightness

 delay(100); // Small delay to smooth the readings
}
```

Code Explanation

- In this Arduino program, a sound sensor is connected to analog pin A3 (SOUND_PIN) and an LED is connected to pin 5 (LED_PIN).
- The setup() function runs once when the Arduino starts. It begins serial communication using Serial.begin(9600) to display sound values and sets the LED pin as an OUTPUT.
- The loop() function runs continuously. It reads the sound level using analogRead() and stores it in soundValue, then prints it on the Serial Monitor.
- Next, the map() function converts the sound value (0–1023) into a range of 0–255. Using analogWrite(), the LED brightness is adjusted based on the sound intensity.
- A small delay() is added for smooth readings.

So overall, the Arduino reads sound intensity and controls the LED brightness accordingly.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Integrated Smart System but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define SOUND_PIN A3   // Sound sensor on A3
#define BUZZER_PIN 5   // Buzzer on D5

void setup() {
  Serial.begin(9600);
  pinMode(BUZZER_PIN, OUTPUT);
}

void loop() {
  int soundValue = analogRead(SOUND_PIN); // Read sound (0–1023)

  Serial.print("Sound value: ");
  Serial.println(soundValue);

  // Trigger buzzer if sound is loud
  if (soundValue > 500) {
    digitalWrite(BUZZER_PIN, HIGH); // Buzzer ON
  } else {
    digitalWrite(BUZZER_PIN, LOW);  // Buzzer OFF
  }

  delay(100);
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Integrated Smart System how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Advanced Integration Model

Class Explanation

An Advanced Integration Model is a system that combines multiple sensors, control units, and logic circuits into a single, efficient setup. It processes various inputs simultaneously to produce intelligent outputs, enabling complex automation, improved accuracy, and advanced functionality in modern electronic applications.

Materials Required

- Arduino with attached shield
- 1 × push button
- 1 × 7 segment display
- RMC wires
- USB cable

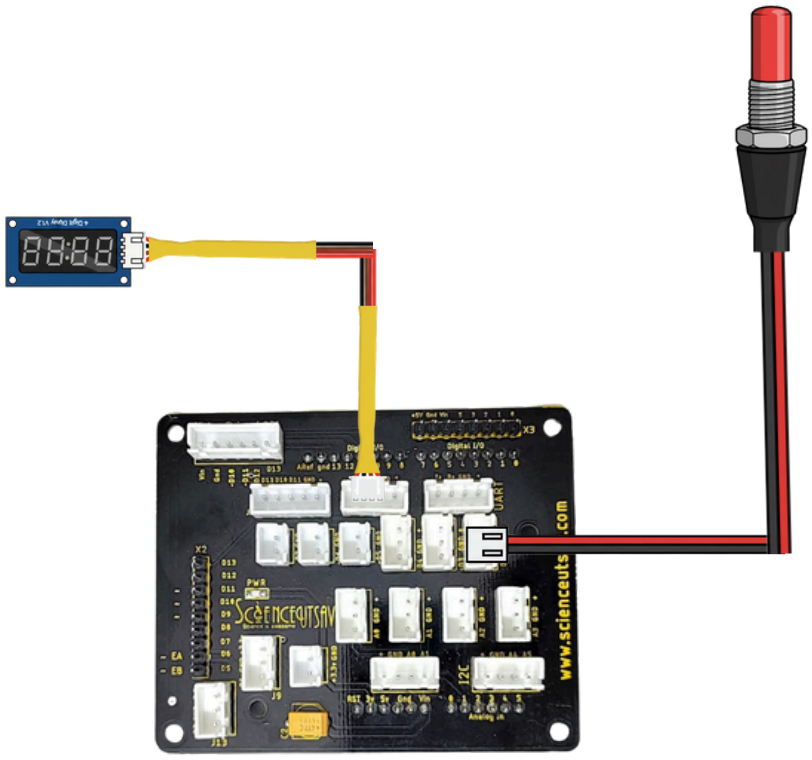
How to Connect

Use RMC wires to connect :

- 1 × push button→ Arduino pin D2 and GND
- 1 × 7 segment display → Arduino pin D8 D9 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#include <TM1637Display.h> // Include library for 4-digit 7-segment display

#define CLK 8 // Clock pin connected to D8
#define DIO 9 // Data pin connected to D9
#define BUTTON_PIN 2 // Push button connected to D2

TM1637Display display(CLK, DIO); // Create display object

int count = 0;     // Variable to store button press count
int lastButtonState = LOW;

void setup() {
 pinMode(BUTTON_PIN, INPUT);  // Set button pin as input
 display.setBrightness(7);   // Set display brightness (0 to 7)
 display.showNumberDec(0);   // Initialize display with 0
}

void loop() {
 int buttonState = digitalRead(BUTTON_PIN);

 // Detect button press (LOW to HIGH transition)
 if (buttonState == HIGH && lastButtonState == LOW) {
  count++;           // Increment count
  if (count > 9999) count = 0; // Reset after 9999
  display.showNumberDec(count); // Display the count
  delay(200);         // Debounce delay
 }

 lastButtonState = buttonState; // Store last button state
}
```

Code Explanation

- In this Arduino program, a push button is connected to pin 5 (BUTTON_PIN) and a TM1637 4-digit display is connected to pins 8 (CLK) and 9 (DIO). The display shows the number of button presses.

- The setup() function runs once when the Arduino starts. It sets the button pin as INPUT, sets the display brightness using setBrightness(), and shows 0 initially on the display.

- The loop() function runs continuously. It reads the button state using digitalRead() and compares it with the previous state stored in lastButtonState.

- When the button is pressed (change from LOW to HIGH), the count value increases by 1. The updated count is displayed using showNumberDec(). If the count exceeds 9999, it resets back to 0.

- A small delay() is used to prevent multiple counts from a single press.

So overall, each button press increases the count and displays it on the screen.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Advanced Integration Model but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#include <TM1637Display.h>

#define CLK 8
#define DIO 9
#define BUTTON_PIN 5

TM1637Display display(CLK, DIO);

int count = 0;
bool running = false;

int lastButtonState = LOW;
unsigned long lastPressTime = 0;
int pressCount = 0;

unsigned long lastCountTime = 0;

void setup() {
  pinMode(BUTTON_PIN, INPUT);
  display.setBrightness(7);
  display.showNumberDec(0);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);

  // Detect button press
  if (buttonState == HIGH && lastButtonState == LOW) {
    pressCount++;
    lastPressTime = millis();
    delay(50);
  }
```

```
  // Double press detection (within 400 ms)
  if (pressCount == 2 && (millis() - lastPressTime) < 400) {
  running = true;
  count = 1; // Start from 1
  display.showNumberDec(count);
  lastCountTime = millis();
  pressCount = 0;
  delay(200);
  }

  // Reset press count if too slow
  if ((millis() - lastPressTime) > 400) {
  pressCount = 0;
  }

  // Auto counting
  if (running) {
  if (millis() - lastCountTime >= 500) {
  count++;
  if (count > 9999) count = 1;

  display.showNumberDec(count);
  lastCountTime = millis();
  }

  // Stop and reset after 3 seconds
  if (millis() - lastPressTime > 3000) {
  running = false;
  count = 0;
  display.showNumberDec(0);
  }
  }

  lastButtonState = buttonState;
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:
the basic idea of Advanced Integration Model how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how sensors work with timing circuits to control outputs for a specific duration in automation and delay-based applications.

- Learn how to combine multiple sensor inputs using logic circuits to solve problems and control outputs effectively.

- Understand how sound sensors are integrated with logic circuits to create automated systems that respond to audio signals.

- Learn how multiple sensors and logic systems are combined to build complex, intelligent electronic systems with improved performance and functionality.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output